

A PROJECT REPORT
on
TinyML-Based Hand Gesture Control Robot

Submitted in partial fulfilment for the completion of course
of
21EEEC305P MICROCONTROLLER
in
ELECTRICAL AND ELECTRONICS ENGINEERING
by

Abde Ali	(RA2311005010054)
Ansh Srivastava	(RA2311005010056)
Lathief Hussain	(RA2311005010058)

Under the guidance of
Dr. R.Narayanamoorthi
(Associate Professor, Department of Electrical and Electronics Engineering)



SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM Nagar, Kattankulathur- 603 203
Chengalpattu District.

May 2026

BONAFIDE CERTIFICATE

Certified that this project report titled “**TinyML-Based Hand Gesture Control Robot**” is the Bonafide work of **Abde Ali (RA2311005010054)**, **Ansh Srivastava (RA2311005010056)**, **Lathief Hussain (RA2311005010058)** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion of this or any other candidate.

Signature of the Guide

Dr. R. Narayanamoorthi

Associate Professor

Department of EEE

SRM IST

ACKNOWLEDGEMENT

We express our sincere gratitude to the **Management of SRM Institute of Science and Technology** for providing us with an excellent academic environment and the necessary infrastructure to successfully carry out our project. We are especially thankful to **Dr. C. Muthamizhchelvan, Vice Chancellor**, for his leadership and commitment to fostering research and innovation. We also extend our heartfelt thanks to **Dr. S. Ponnusamy, Registrar**, for ensuring seamless academic processes, and to **Dr. Leenus Jesu Martin M, Dean (CET)**, for his continuous support and encouragement towards student projects and academic growth.

We would like to thank our beloved **Dr. K. Vijayakumar, Chairperson, School of Electrical and Electronics Engineering**, for his continued encouragement throughout the course of study. We express our sincere gratitude to **Dr. R. Sridhar, Head of the Department**, for his support towards the successful completion of this project. We place on record our sincere thanks to **Dr.J.Preetha Roselyn**, our **academic advisor** for her consistent mentoring. We also thank our **faculty advisor, Dr.C.Nithya** for her valuable guidance and continuous support.

We extend our sincere gratitude to **Dr. R.Narayanamoorthi**, our **project guide**, for his valuable insights, dedicated mentorship, and constant encouragement during the course of this work.

We would also like to express heartfelt thanks to all our project evaluators for their constructive feedback and insightful suggestions which helped us refine our work further.

Abde Ali

Ansh Srivastava

Lathief Hussain

ABSTRACT

This report presents the design and implementation of a TinyML-based hand gesture-controlled robot that enables intuitive, controller-free robotic navigation using natural hand movements. The system leverages an MPU-6050 six-axis Inertial Measurement Unit (IMU) sensor interfaced with an ESP32 microcontroller to capture real-time motion data from the user's hand. A neural network model trained on the Edge Impulse platform classifies five distinct gestures — Forward, Backward, Left, Right, and Still — directly on the ESP32 using a quantized TinyML model, enabling on-device inference without any cloud dependency.

The classified gesture is transmitted wirelessly via NRF24L01 radio modules to a receiver unit built around an Arduino Nano. Upon receiving the gesture command, the Arduino Nano drives two N20 micro gear motors through a DRV8833 dual motor driver to physically manoeuvre the robot in the corresponding direction. Simultaneously, an OLED display mounted on the receiver renders expressive animated face graphics that provide intuitive visual feedback of the current robot state.

A total of 125 motion samples were collected — 25 samples per gesture class — and used to train a neural network classifier on Edge Impulse. The trained model achieved a validation accuracy of 97.8% with a loss of 0.20 and an Area Under the ROC Curve of 1.00, demonstrating strong generalization across all five gesture classes. The model was exported as an Arduino-compatible library and integrated directly into the ESP32 firmware.

This project demonstrates the practical viability of TinyML for real-time gesture recognition on resource-constrained microcontrollers and presents an effective, low-cost approach to natural human-robot interaction suitable for educational, assistive, and hobbyist robotics applications.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	iv
	LIST OF FIGURES	vii
	LIST OF TABLES	viii
1	INTRODUCTION	
	1.1 GENERAL	1
	1.2 MOTIVATION	2
	1.3 LITERATURE SURVEY	2
	1.4 PROBLEM FORMULATION	4
2	1.5 OBJECTIVES	5
	SYSTEM DESIGN	
	2.1 INTRODUCTION	7
	2.2 SYSTEM ARCHITECTURE	7
	2.3 CIRCUIT WORKING	8
3	2.4 CONCLUSION	11
	TINYML MODEL TRAINING	
	3.1 TINYML OVERVIEW	12
	3.2 DATA ACQUISITION	12
	3.3 IMPLUSE CREATION	14
	3.4 SPECTURAL FEATURES EXTRACTION	15
	3.5 CLASSIFICATION	17
4	3.6 DEPLOYMENT	20
	HARDWARE IMPLEMENTATION	
	4.1 HARDWARE SETUP	21
5	4.2 STANDARD COMPLIANCE	24
	CONCLUSION	
	5.1 CONCLUSION	26
6	5.2 FUTURE SCOPE	26
	REFERENCES	28

LIST OF FIGURES

FIGURE NO.	FIGURE	PAGE NO.
1.1	Block diagram of overall system	6
2.1	Transmitter Circuit Diagram	8
2.2	Receiver Circuit Diagram	9
3.1	Edge Impulse — Data Acquisition Screen	13
3.2	Edge Impulse — Impulse Creation Screen	15
3.3	Edge Impulse — Spectral Features Extraction Screen	17
3.4	Edge Impulse — Classifier Training Results and Confusion Matrix	18
4.1	Hardware Setup	21

LIST OF TABLES

Table No.	Table	Page No.
2.1	Pin Mapping — Transmitter Circuit	9
2.2	Pin Mapping — Receiver Circuit	10
3.1	Dataset Summary — Gesture Classes and Sample Count	14
3.2	Impulse Design Configuration	14
3.3	Spectral Analysis Parameters	16
3.4	Neural Network Training Configuration	18
3.5	Model Performance Metrics	19
3.6	Confusion Matrix Results	19
4.1	Transmitter Hardware Components and Specifications	21
4.2	Receiver Hardware Components and Specifications	23

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Robotics has evolved significantly over the past few decades, transitioning from purely industrial applications to consumer, educational, and assistive contexts. One of the most persistent challenges in this evolution is developing intuitive and accessible control interfaces. Conventional robotic control mechanisms — joysticks, keypads, touchscreens, and dedicated remote controllers — impose a learning curve and create a physical and cognitive separation between the user and the machine.

The emergence of TinyML, which refers to the deployment of trained machine learning models on ultra-low-power microcontrollers, offers a compelling solution to this challenge. By embedding intelligence directly into edge hardware, it becomes possible to interpret complex human inputs such as gestures, speech, or facial expressions in real time, without any reliance on cloud servers or high-performance computing resources.

This project presents a hand gesture-controlled robot powered by TinyML. An MPU-6050 IMU sensor worn on the user's hand captures six-axis motion data encompassing three-axis linear acceleration and three-axis angular velocity. An ESP32 microcontroller runs an on-device neural network model to classify the captured motion into one of five gesture categories: Forward, Backward, Left, Right, and Still. The inferred gesture label is wirelessly transmitted to a receiver module via NRF24L01 2.4GHz radio communication. On the receiver side, an Arduino Nano interprets the incoming command and actuates two N20 micro gear motors through a DRV8833 motor driver to move the robot in the commanded direction. An OLED panel on the receiver simultaneously renders expressive graphical faces to provide visual feedback of the robot's current state.

The entire intelligence pipeline — from data collection and model training on the Edge Impulse platform to deployment as an Arduino library on the ESP32 — is documented in this report, along with hardware design, circuit implementation, and performance evaluation.

1.2 MOTIVATION

The motivation behind this project stems from multiple converging interests in embedded systems, machine learning, and human-robot interaction.

Traditional gesture recognition systems often depend on computer vision, requiring cameras, image processing pipelines, and significant computational resources. These requirements make such systems costly, power-hungry, and impractical for lightweight embedded applications. IMU-based gesture recognition, by contrast, is compact, energy-efficient, and highly robust to environmental conditions such as lighting variation or occlusion.

Furthermore, the maturation of TinyML toolchains — particularly Edge Impulse and TensorFlow Lite for Microcontrollers — has dramatically lowered the barrier to deploying machine learning models on microcontrollers. This creates an exciting opportunity to bring machine intelligence into everyday embedded projects that previously could only rely on hardcoded logic.

From a human-robot interaction perspective, gesture-based control is inherently more natural and expressive than button-based control. A user can command a robot to move simply by tilting their hand in the desired direction, which aligns with natural motor intuition and reduces the cognitive load of operating the system. This has meaningful implications for accessibility, particularly for users who may find traditional controllers difficult to operate.

The combination of these factors — accessible TinyML tooling, affordable IMU hardware, and the appeal of natural gesture interfaces — motivated the design of this project as a practical demonstration of what modern embedded machine learning can achieve.

1.3 LITERATURE SURVEY

- **TinySenseNet: A Lightweight sEMG-IMU Fusion Network Using TinyML for Mechanical Arm Control in Low-Resource Settings**

Authors: Md. Rifat Aknda et al.

Summary: TinySenseNet is a lightweight deep learning model for real-time robotic arm control using multimodal sEMG and IMU signals, optimized for embedded deployment with 150ms input windows. The architecture uses depth wise 1D convolutions and an attention mechanism for efficient feature extraction. A standout innovation is the cross-modal delay feature that captures the timing gap between muscle activation and physical motion, improving real-world robustness. With a model size of just 1.46KB and 23.8ms inference latency, it achieved 97.2% accuracy for able-bodied users and 95.7% for

amputees, outperforming CNN, LSTM, and SVM baselines. The hardware platform used an ESP32 microcontroller paired with a Myo Armband and servo-driven robotic arm. [1]

- **Microcontroller Unit-Based Gesture Recognition System**

Authors: Jakub Grabarczyk and Agnieszka Lazarowska Vol. 13, 2025.

Summary: This paper describes a microcontroller-based gesture recognition system using hand gestures and machine learning to control a UAV. The system uses a neural network model running on a Raspberry Pi Pico with an IMU sensor and a radio communication module for wireless command transmission. The paper provides an honest technical analysis of the tradeoffs between deploying models on constrained microcontrollers versus more powerful processors, finding that microcontrollers are entirely viable for real-time gesture-based control when the model is appropriately optimized. The work demonstrates a practical end-to-end pipeline from gesture capture to wireless robot actuation on minimal hardware. [2]

- **Gesture Controlled Robot for Human Detection**

Authors: Athira T.S., Honey Manoj, Vishnu K. Menon, Srilekshmi M.

Summary: This paper presents a hand gesture-controlled robot for locating survivors in collapsed buildings, where the robot is guided by hand gestures transmitted wirelessly from a control unit, while a PIR sensor on the robot detects trapped humans and relays alerts back to the operator. The system established the two-unit transmitter-receiver architecture as a natural design pattern for gesture-controlled robots, demonstrating that untrained users can effectively navigate a robot in hazardous environments using intuitive hand gestures. It remains one of the clearer early examples of applying gesture robotics to a humanitarian purpose. [3]

1.3.1 INFERENCE FROM LITERATURE SURVEY

The three papers collectively point toward several conclusions that directly shaped the design of this project.

The most consistent finding across all works is that IMU sensors are a reliable, low-cost, and practical foundation for gesture recognition. Every paper — whether using IMU alone or in combination with sEMG — confirms that six-axis motion data carries enough information to meaningfully classify directional hand gestures. This validated the choice

of the MPU-6050 in this project with confidence.

Equally important is the confirmation that TinyML inference on microcontrollers is not just theoretically possible but practically accurate. TinySenseNet achieved above 97% accuracy in a model smaller than 2KB. The UAV control paper deployed a full neural network on a Raspberry Pi Pico. Both show that resource constraints do not have to mean accuracy compromises when the model is well-designed and the feature engineering is appropriate. The 97.8% accuracy achieved by this project's Edge Impulse classifier sits comfortably in line with what the literature considers achievable.

The two-unit wireless architecture — a wearable sensing and inference unit on the user, and a separate actuator unit on the robot — appears independently across the ICICNIS paper and this project, which suggests it is the natural and optimal structure for this class of system. The intelligence belongs near the sensor, and the motors belong on the robot. Finally, TinySenseNet raises an important point about confidence-based actuation: a robot should only respond when the model is sufficiently certain of its prediction. This is a practical safety consideration that this project identifies as a meaningful direction for future improvement, particularly as the system is refined toward more demanding real-world applications.

Together, these papers provide both technical validation and design inspiration for a gesture-controlled robot built on TinyML, IMU sensing, and wireless communication — the exact combination this project implements.

1.4 PROBLEM FORMULATION

Conventional robotic remote-control systems present several limitations that reduce their practicality and accessibility.

First, they require a dedicated physical controller, which must be learned, carried, and maintained separately from the robot itself. Second, most simple remote-control systems are purely reactive and carry no embedded intelligence — they transmit raw input signals without any interpretation or context awareness. Third, systems that do incorporate gesture recognition often depend on cameras or cloud-based inference, introducing latency, connectivity requirements, privacy concerns, and increased cost.

The specific problem this project addresses can be stated as follows:

How can a low-cost, self-contained robotic system be designed that interprets natural

hand gestures in real time using an on-device machine learning model, without any cloud connectivity, and translates those gestures into directional motor commands with simultaneous visual feedback?

The constraints of the problem include the need to operate on microcontrollers with limited flash and RAM, achieve real-time inference latency suitable for responsive robot control, maintain wireless communication reliability over practical indoor distances, and keep the overall bill of materials affordable and reproducible.

1.5 OBJECTIVES

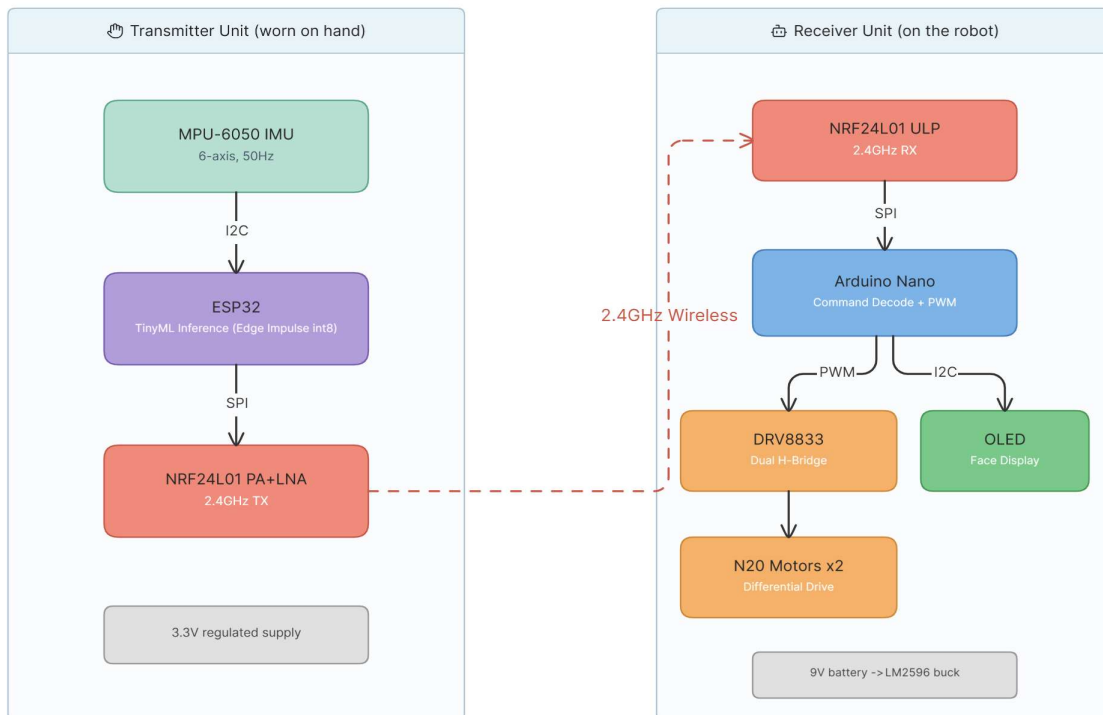
- To design and build a wearable transmitter unit incorporating an MPU-6050 IMU sensor and an ESP32 microcontroller capable of capturing and classifying hand gestures in real time using a TinyML model.
- To collect a labelled dataset of 125 IMU motion samples across five gesture classes — Forward, Backward, Left, Right, and Still — using the Edge Impulse data acquisition pipeline.
- To design and train a spectral feature-based neural network classifier on Edge Impulse that achieves high accuracy on the collected gesture dataset.
- To deploy the trained model as a quantized int8 Arduino library onto the ESP32, enabling on-device inference without cloud dependency.
- To design and build a receiver unit based on Arduino Nano that receives classified gesture labels wirelessly via NRF24L01 and drives two N20 motors through a DRV8833 motor driver accordingly.
- To integrate an OLED display into the receiver that renders contextual animated face expressions corresponding to each robot movement state.
- To validate the complete end-to-end system through testing and evaluate its performance in terms of classification accuracy, response latency, and movement correctness.

1.6 BLOCK DIAGRAM

The block diagram illustrates the complete signal flow of the TinyML-based hand gesture-controlled robot across its two hardware subsystems. On the transmitter side, the MPU-6050 IMU sensor continuously streams six-axis motion data over the I2C bus to the ESP32 microcontroller at a rate of 50Hz. The ESP32 runs the Edge Impulse-generated TinyML model — quantized to int8 precision — to classify the incoming motion data into one of five gesture labels in real time. The classified label is then passed over SPI to the NRF24L01 PA+LNA radio module, which broadcasts it wirelessly over the 2.4GHz ISM band. The entire transmitter unit runs on a regulated 3.3V supply.

On the receiver side, the NRF24L01 ultra-low-power module continuously listens for incoming packets over the same 2.4GHz channel. Upon reception, the data is passed via SPI to the Arduino Nano, which decodes the gesture label and takes two simultaneous actions: it sends PWM signals to the DRV8833 dual H-bridge motor driver to actuate the two N20 micro gear motors in the direction corresponding to the gesture, and it sends display commands over I2C to the OLED panel to render the appropriate animated face expression. The receiver unit is powered from a 9V battery regulated down by an LM2596 buck converter.

Figure 1.1. Block Diagram



CHAPTER 2

SYSTEM DESIGN

2.1 INTRODUCTION

This chapter presents the complete system design of the TinyML-based hand gesture-controlled robot. The system is composed of two discrete hardware units — the transmitter and the receiver — that communicate wirelessly. The transmitter captures and classifies hand gestures; the receiver actuates the robot motors and provides visual feedback. The design philosophy prioritizes low cost, low power consumption, modularity, and real-time performance.

2.2 SYSTEM ARCHITECTURE

The overall system architecture follows a sense-infer-transmit-actuate pipeline, as described below.

1. **Transmitter Side:** The MPU-6050 IMU sensor continuously reads six-axis motion data from the user's hand at a sampling frequency of 50Hz. This data is fed into a pre-loaded TinyML model running on the ESP32. The model processes a 2-second window of sensor data, extracts spectral features, and outputs a gesture classification label. The ESP32 then encodes this label and transmits it wirelessly via the NRF24L01 PA+LNA radio module operating at 2.4GHz.
2. **Receiver Side:** The NRF24L01 ultra-low-power module on the receiver side listens continuously for incoming packets. Upon receiving a gesture label, the Arduino Nano decodes the command and sends appropriate PWM signals to the DRV8833 dual H-bridge motor driver, which controls the speed and direction of two N20 micro gear motors. Simultaneously, the Arduino Nano updates the OLED display with a corresponding animated face expression — for example, a forward-looking face for Forward, a surprised face for Backward, and so on.

2.3 CIRCUIT WORKING

2.3.1 Transmitter Circuit

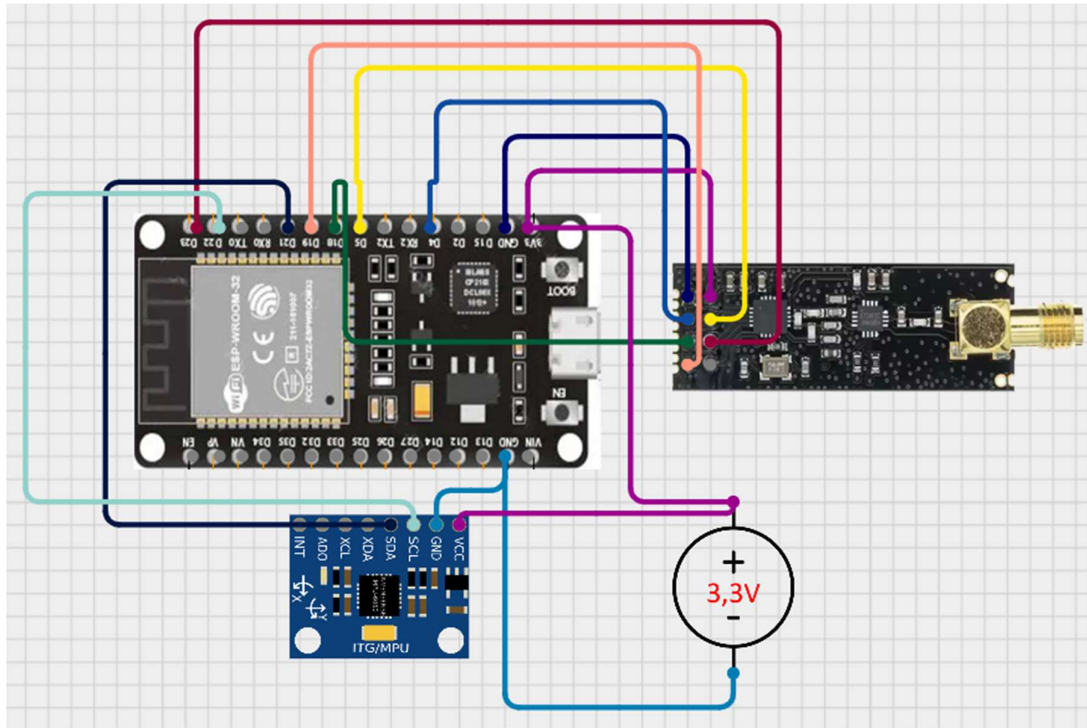


Figure 2.1. Transmitter Circuit Diagram

The transmitter circuit is built around the ESP32 microcontroller, which serves as the central processing unit. The MPU-6050 sensor communicates with the ESP32 over the I2C protocol. The sensor's SDA and SCL lines are connected to the designated I2C pins of the ESP32, with the sensor powered at 3.3V. The MPU-6050 simultaneously provides accelerometer data ($\pm 8g$ range) and gyroscope data ($\pm 500^\circ/s$ range) across all three axes, yielding six channels of motion information.

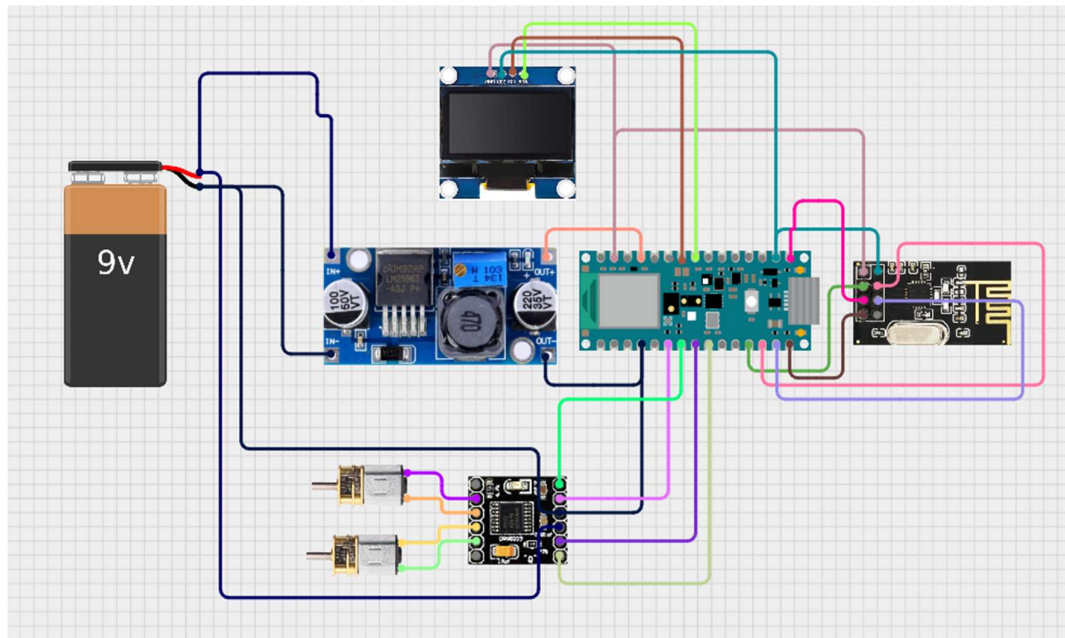
The NRF24L01 with external antenna module communicates with the ESP32 via the SPI protocol. The CE and CSN control pins are connected to configurable GPIO pins on the ESP32. The module is powered at 3.3V with a decoupling capacitor placed close to the VCC pin to ensure power stability during burst transmission events.

The ESP32 runs the full inference pipeline: it reads raw IMU data at 50Hz, buffers a 2-second window (100 samples per channel), passes the buffer through the Edge Impulse-generated model library, and receives a predicted gesture label. This label is packed into a single byte and transmitted over NRF24L01 to the receiver.

Table 2.1. Pin Mapping — Transmitter Circuit

Component	Signal	ESP32 Pin
MPU-6050	SDA	GPIO 21
MPU-6050	SCL	GPIO 22
MPU-6050	VCC	3.3V
MPU-6050	GND	GND
NRF24L01	MOSI	GPIO 23
NRF24L01	MISO	GPIO 19
NRF24L01	SCK	GPIO 18
NRF24L01	CSN	GPIO 5
NRF24L01	CE	GPIO 4
NRF24L01	VCC	3.3V
NRF24L01	GND	GND

2.3.2 Receiver Circuit

**Figure 2.2. Receiver Circuit Diagram**

The receiver circuit centres on the Arduino Nano. The NRF24L01 ultra-low-power module is connected to the Arduino via SPI, listening for incoming gesture labels broadcast by the transmitter. Upon reception, the Arduino decodes the label byte and maps it to one of five motor control behaviours.

The DRV8833 dual H-bridge motor driver receives PWM signals from the Arduino Nano on its IN1, IN2, IN3, and IN4 input pins, which correspond to the two channels controlling the left and right N20 motors respectively. The DRV8833 operates at the voltage supplied by the buck-converted output (approximately 3.3–5V), which is well within the operating range of the N20 motors.

The OLED display (128×64 pixels, I2C interface) is connected to the Arduino Nano's I2C pins. The Adafruit SSD1306 library is used to render custom bitmap face graphics for each gesture state. The entire receiver unit is powered from a 9V battery stepped down by an LM2596 adjustable buck converter to provide a stable regulated supply.

Table 2.2. Pin Mapping — Receiver Circuit

Component	Signal	Arduino Nano Pin
NRF24L01	MOSI	D11
NRF24L01	MISO	D12
NRF24L01	SCK	D13
NRF24L01	CSN	D10
NRF24L01	CE	D9
DRV8833	IN1	D5
DRV8833	IN2	D6
DRV8833	IN3	D7
DRV8833	IN4	D8
OLED	SDA	A4
OLED	SCL	A5
OLED	VCC	3.3V

2.1 CONCLUSION

This chapter presented the complete system architecture and circuit design of both the transmitter and receiver units. The transmitter captures IMU data, performs on-device TinyML inference, and wirelessly transmits the gesture label, while the receiver decodes and acts on that label through motor actuation and visual feedback. The modular two-unit design ensures clean separation of responsibilities and allows each subsystem to be developed and tested independently. The following chapter details the TinyML model training pipeline that forms the intelligence core of this system.

CHAPTER 3

TINYML MODEL TRAINING

3.1 TINYML OVERVIEW

TinyML is a branch of machine learning focused on deploying trained inference models onto microcontrollers and other ultra-low-power embedded devices. Unlike conventional machine learning workflows that execute on servers or GPUs, TinyML models operate locally on the device itself, consuming milliwatts of power and responding within milliseconds.

The key enablers of TinyML are model quantization and optimized inference engines. Quantization reduces the numerical precision of model weights from 32-bit floating point to 8-bit integers (int8), dramatically reducing model size and computational complexity with only marginal impact on accuracy. TensorFlow Lite for Microcontrollers provides the optimized runtime environment for executing such models on constrained hardware.

Edge Impulse is the platform used in this project to manage the entire TinyML pipeline — from raw data collection to model training and final deployment. It abstracts much of the complexity involved in feature engineering and model optimization, while still exposing sufficient configurability for technical customization.

The model training pipeline followed in this project consists of four major stages: Data Acquisition, Impulse Creation, Spectral Feature Extraction, and Classification. Each stage is described in detail in the following sections, with reference to the corresponding screenshots captured from Edge Impulse.

3.2 DATA ACQUISITION

Data acquisition is the foundational step of any machine learning pipeline. The quality, diversity, and balance of the collected dataset directly determine the capability of the trained model to generalize to real-world inputs.

In this project, the MPU-6050 sensor mounted on the user's hand was used to collect six-channel IMU data — accX, accY, accZ, gyroX, gyroY, and gyroZ — corresponding to

five distinct hand gesture classes. Data was streamed from the ESP32 to the Edge Impulse platform using the Edge Impulse data forwarder utility over a serial connection.

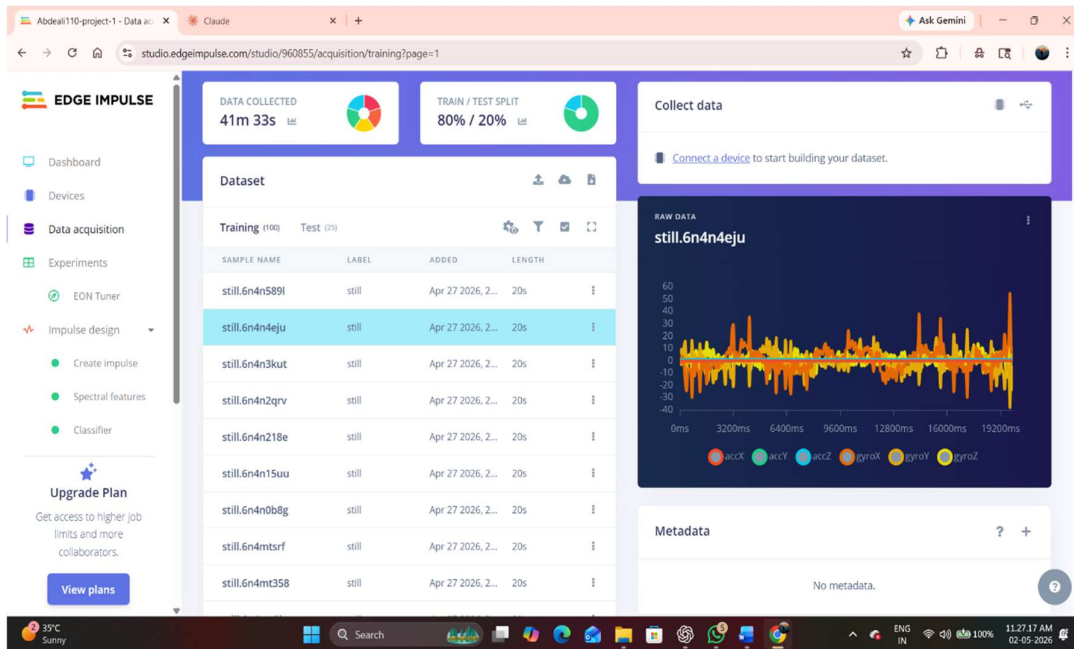


Figure 3.1. Edge Impulse — Data Acquisition Screen

As seen in the data acquisition screen (Image 2), the dataset totals 41 minutes and 33 seconds of recorded IMU motion data. The train/test split was configured at 80%/20%, resulting in 100 training samples and 25 test samples. Each sample was recorded at 50Hz, producing 1,000 data points per channel per 20-second sample.

The raw data visualization shown on the right side of the acquisition screen displays the six sensor channels for a "still" gesture sample. The signal exhibits the characteristic low-amplitude, high-frequency noise pattern expected when the hand is held stationary, confirming correct sensor operation and appropriate data labelling.

Each gesture was performed with consistent, deliberate motion to ensure representative and clean samples. For the Forward gesture, the user tilted the hand forward along the pitch axis. For Backward, the hand was tilted in the opposite direction. Left and Right gestures involved rolling the hand laterally in the respective directions. The Still gesture was captured with the hand held flat and stationary. Care was taken to vary the speed and intensity of gestures slightly across samples to improve model robustness.

Table 3.1. Dataset Summary — Gesture Classes and Sample Count

Gesture Class	Samples Collected	Duration per Sample	Total Duration
Forward	25	20 seconds	~8 min 20 sec
Backward	25	20 seconds	~8 min 20 sec
Left	25	20 seconds	~8 min 20 sec
Right	25	20 seconds	~8 min 20 sec
Still	25	20 seconds	~8 min 20 sec
Total	125	—	~41 min 33 sec

3.3 IMPLUSE CREATION

The impulse in Edge Impulse terminology refers to the complete processing pipeline that transforms raw sensor data into a classified output. It consists of three sequential blocks: the input data block, the signal processing block, and the learning block.

The input block is configured to accept six-axis time series data from the MPU-6050 sensor. The following parameters were set:

Table 3.2. Impulse Design Configuration

Parameter	Value
Input Axes	accX, accY, accZ, gyroX, gyroY, gyroZ
Window Size	2000 ms
Window Increase (Stride)	200 ms
Frequency	50 Hz
Zero-pad Data	Enabled
Train on Data Subset	100%

The window size of 2000 ms means the model processes 2 seconds of sensor data at a time, which was found to be sufficient to fully capture the temporal extent of each gesture. The stride of 200 ms means consecutive windows overlap significantly, increasing the number of training samples derived from each recording and improving temporal coverage.

Zero-padding is enabled to handle cases where the tail end of a recording is shorter than the full window size, ensuring no data is discarded.

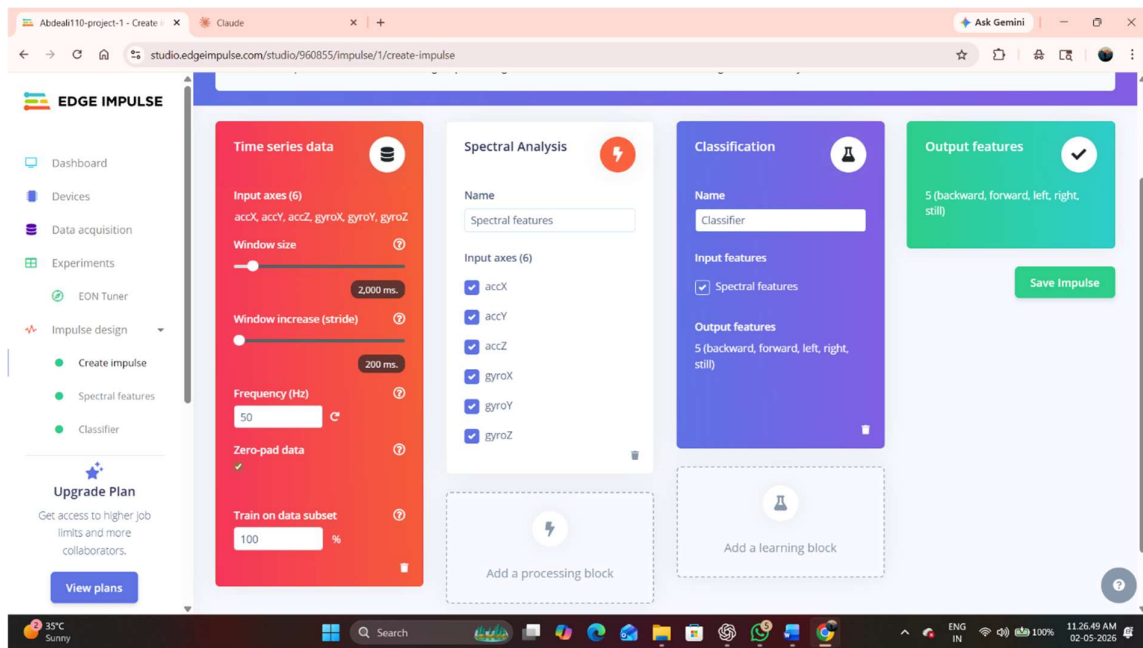


Figure 3.2. Edge Impulse — Impulse Creation Screen

The signal processing block applies Spectral Analysis to the raw time series data, transforming it into frequency-domain features. All six input axes are included in the processing block, as each axis contributes unique motion information.

The learning block is a neural network classifier named "Classifier." Its input is the Spectral Features output from the processing block, and its output is a probability distribution across five gesture classes: backward, forward, left, right, and still. The output features block on the right of the impulse screen confirms these five output classes.

3.4 SPECTRAL FEATURES EXTRACTION

Raw IMU time series data contains significant redundancy and noise that can hamper neural network training if used directly. Spectral feature extraction transforms the time-domain signal into a compact frequency-domain representation that highlights the periodic

and oscillatory characteristics of each gesture, making the classification task considerably easier for the downstream neural network.

No filtering was applied at this stage since the raw IMU data was already sufficiently clean for the gesture classes being classified, and the FFT-based analysis inherently provides some noise separation through frequency binning.

Table 3.3. Spectral Analysis Parameters

Parameter	Value
Analysis Type	FFT (Fast Fourier Transform)
FFT Length	16
Take Log of Spectrum	Enabled
Overlap FFT Frames	Enabled
Improve Low Frequency Resolution	Disabled

The FFT length of 16 points was selected to provide a compact spectral representation while preserving the key frequency components that differentiate the five gestures.

Taking the log of the spectrum compresses the dynamic range of spectral power values, making the features more numerically stable for neural network training. Overlapping FFT frames increases the effective number of spectral snapshots per window, improving feature richness.

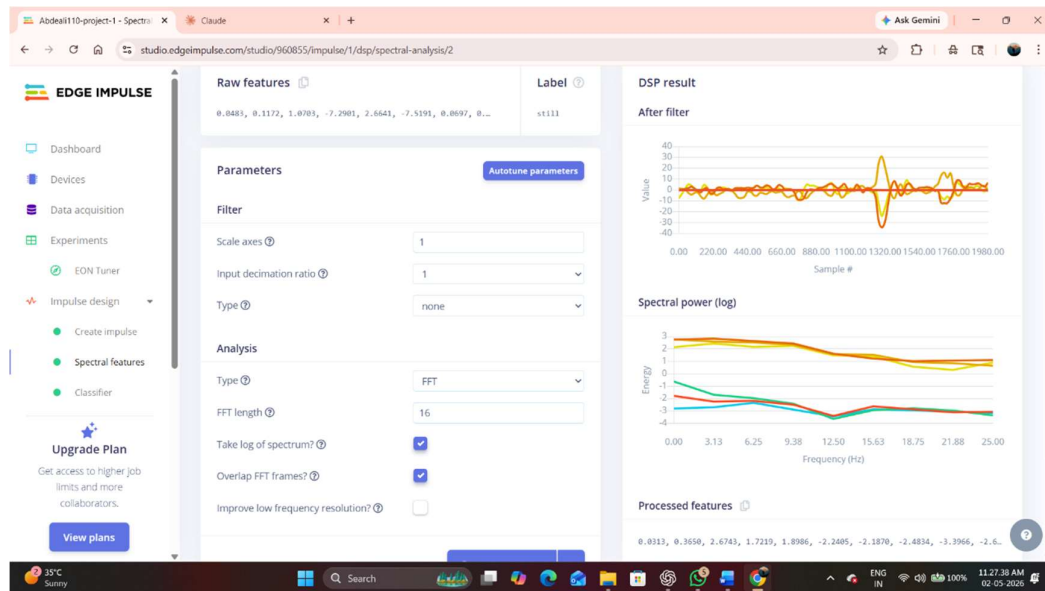


Figure 3.3. Edge Impulse — Spectral Features Extraction Screen

The DSP result panel on the right side of the screen figure 3.3 shows the output of the spectral analysis for a "still" gesture sample. The "After filter" graph shows the time-domain signal after processing — the low-amplitude oscillations are characteristic of a stationary hand. The "Spectral power (log)" graph below it shows the frequency content of each sensor axis, with energy concentrated in the low-frequency range as expected for a still posture.

The processed features for this sample begin with values such as 0.0313, 0.3650, 2.6743, 1.7219, 1.8986, -2.2405... These 78 numerical values constitute the complete feature vector that is passed to the neural network classifier as input.

The total number of input features to the classifier is 78, derived from applying the FFT-based spectral analysis across all six sensor axes with the configured parameters

3.5 CLASSIFICATION

The classification stage involves designing, training, and evaluating the neural network that maps the 78-dimensional spectral feature vector to one of five gesture class labels. Edge Impulse's Keras-based neural network designer was used for this purpose, as seen in the Classifier screen (figure 3.4).

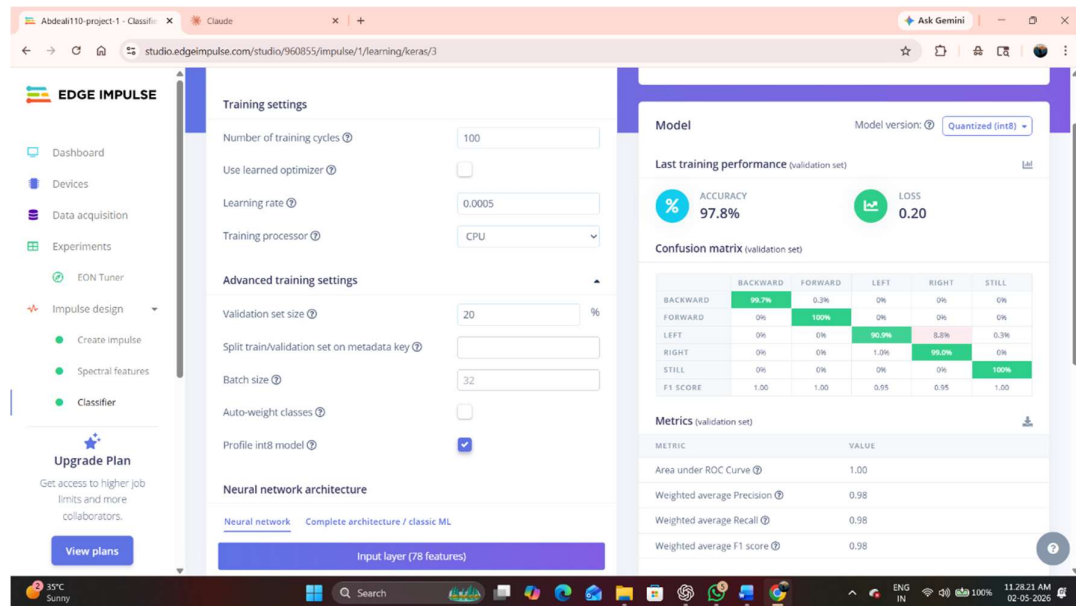


Figure 3.4. Edge Impulse — Classifier Training Results and Confusion Matrix

Table 3.4. Neural Network Training Configuration

Parameter	Value
Number of Training Cycles	100
Learning Rate	0.0005
Use Learned Optimizer	Disabled
Training Processor	CPU
Validation Set Size	20%
Batch Size	32
Auto-weight Classes	Disabled
Profile int8 Model	Enabled

The neural network takes 78 input features and passes them through a series of fully connected (dense) layers. The architecture begins with an input layer of 78 features, followed by hidden dense layers with ReLU activation functions, and terminates with a SoftMax output layer producing probability scores across the five gesture classes. The int8 quantization profile was enabled to prepare the model for deployment on the memory-constrained ESP32.

The model was trained on the 80% training split and validated on the 20% validation split. The final training performance is summarized below:

Table 3.5. Model Performance Metrics

Metric	Value
Validation Accuracy	97.8%
Validation Loss	0.20
Area Under ROC Curve	1.00
Weighted Average Precision	0.98
Weighted Average Recall	0.98
Weighted Average F1 Score	0.98

The confusion matrix from the validation set provides a class-wise breakdown of classification performance:

Table 3.6. Confusion Matrix Results

<i>Predicted →</i>	<i>BACKWARD</i>	<i>FORWARD</i>	<i>LEFT</i>	<i>RIGHT</i>	<i>STILL</i>
<i>BACKWARD</i>	99.7%	0.3%	0%	0%	0%
<i>FORWARD</i>	0%	100%	0%	0%	0%
<i>LEFT</i>	0%	0%	90.9%	8.8%	0.3%
<i>RIGHT</i>	0%	1.0%	0%	99.0%	0%
<i>STILL</i>	0%	0%	0%	0%	100%
<i>F1 Score</i>	1.00	1.00	0.95	0.95	1.00

The confusion matrix reveals excellent classification performance across all five classes. Four of the five classes achieve perfect or near-perfect classification. The minor confusion between Left and Right gestures (8.8% of Left samples misclassified as Right) is attributable to the inherent

similarity in the rotational motion profiles of lateral gestures, and represents an area for potential improvement through additional data collection or augmentation.

The F1 scores of 1.00 for Backward, Forward, and Still confirm flawless discrimination of these classes. Left and right achieve F1 scores of 0.95 each, which remains highly acceptable for a real-time gesture control application.

3.6 DEPLOYMENT

After training and validation, the model was exported from Edge Impulse as an Arduino library. This export packages the quantized int8 neural network model along with the complete signal processing and inference pipeline into a self-contained .zip library that can be directly imported into the Arduino IDE.

Deployment Steps

Step 1: Export from Edge Impulse: In the Edge Impulse Studio, the Deployment tab was used. "Arduino library" was selected as the export target. Edge Impulse compiled the trained model, applied full int8 quantization, and generated a downloadable .zip file containing all required header files, source files, and the TensorFlow Lite for Microcontrollers inference engine.

Step 2: Import into Arduino IDE: The downloaded .zip library was imported into the Arduino IDE via Sketch → Include Library → Add .ZIP Library. This made the Edge Impulse library available for inclusion in the ESP32 sketch.

Step 3: Integration into ESP32 Sketch: The ESP32 Arduino sketch includes the Edge Impulse library header, initializes the IMU sensor and NRF24L01 module, and implements a continuous inference loop. In each loop iteration, the sketch collects a 2-second window of IMU data, passes it through the Edge Impulse signal processing and inference pipeline, reads the highest-confidence output label, and transmits the corresponding gesture command byte via NRF24L01.

Step 4: Model Size and Resource Utilization: The quantized int8 model is sufficiently compact to fit within the ESP32's flash memory, leaving ample space for the application firmware. The inference time on the ESP32 is within acceptable bounds for real-time gesture recognition at the 200ms stride interval.

CHAPTER 4

HARDWARE IMPLEMENTATION

4.1 HARDWARE SETUP

This chapter details the physical hardware components used in both the transmitter and receiver units, their technical specifications, and the rationale for their selection.

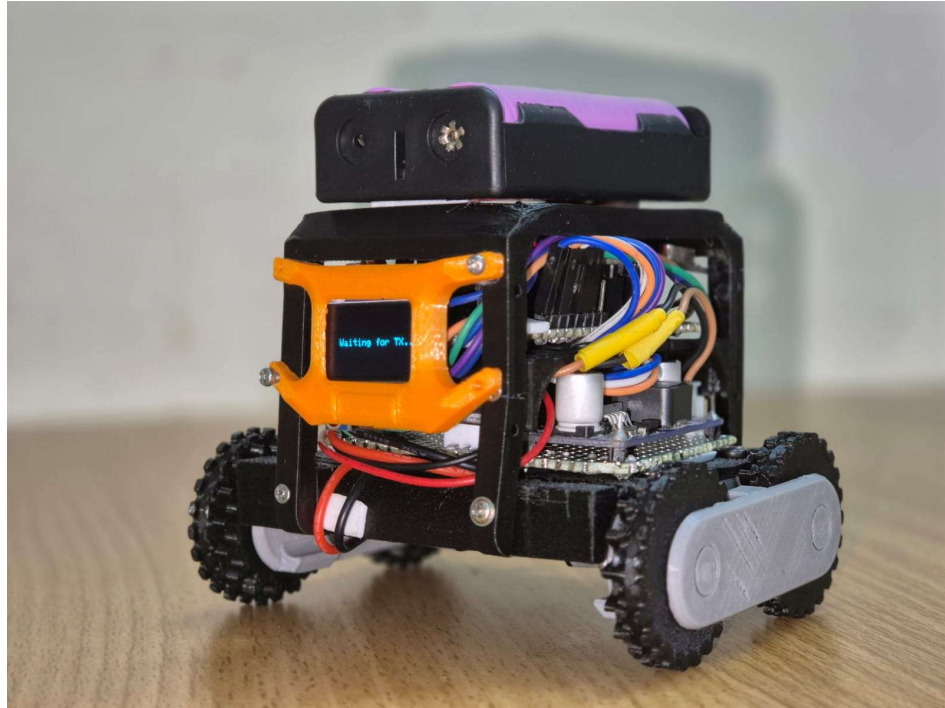


Figure 4.1. Hardware Setup

4.1.1 Transmitter Components

Table 4.1. Transmitter Hardware Components and Specifications

Component	Specification	Purpose
ESP32 (WROOM-32)	Dual-core 240MHz, 520KB SRAM, 4MB Flash, Wi-Fi + BT	Main processing unit, TinyML inference

MPU-6050	3-axis accelerometer ($\pm 8g$), 3-axis gyroscope ($\pm 500^\circ/s$), I2C	Gesture motion capture
NRF24L01 + PA + LNA	2.4GHz, SPI, up to 1km range with antenna	Wireless gesture label transmission
3.3V Power Supply	Regulated 3.3V	Powers ESP32 and peripherals

- **ESP32:** The ESP32 was chosen as the primary microcontroller for the transmitter due to its dual-core processing capability, generous flash memory (4MB), and compatibility with TensorFlow Lite for Microcontrollers. Its support for the Arduino framework allowed seamless integration of the Edge Impulse-generated library. The 240MHz clock speed ensures that the inference pipeline completes well within the 200ms stride window.
- **MPU-6050:** The MPU-6050 is a 6-DoF IMU that combines a 3-axis MEMS accelerometer and a 3-axis MEMS gyroscope in a single package. The accelerometer captures the linear tilt orientation of the hand, while the gyroscope captures rotational velocity. Together, these six channels provide a rich representation of hand motion that allows the model to reliably distinguish between the five gesture classes. The I2C interface simplifies wiring, requiring only two signal lines to the ESP32.
- **NRF24L01 PA+LNA:** The version with an integrated power amplifier (PA) and low-noise amplifier (LNA) was selected for the transmitter to maximize transmission range and signal reliability in indoor environments. The module operates at 2.4GHz, communicates via SPI, and supports data rates up to 2Mbps, making it more than adequate for transmitting small gesture label packets.

4.1.2 Receiver Components

- **Arduino Nano:** The Arduino Nano was selected for the receiver side due to its compact form factor, sufficient I/O pins for interfacing all required peripherals, and compatibility with the RF24 and Adafruit SSD1306 libraries. Since the receiver does not need to run machine learning inference, the ATmega328P's modest processing

capability is entirely sufficient for command decoding, PWM generation, and OLED rendering.

- **DRV8833:** The DRV8833 dual H-bridge motor driver was chosen for its small package, low operating voltage range (compatible with N20 motors), built-in current limiting, and thermal shutdown protection. Its two independent H-bridge channels allow independent control of both N20 motors, enabling differential steering for turning manoeuvres.
- **N20 Micro Gear Motor:** The N20 is a miniature DC gear motor widely used in small robot applications. Its compact dimensions and high gear reduction ratio make it ideal for low-speed, high-torque robotic wheel drive applications. Two motors provide differential drive capability, allowing the robot to turn left or right by varying the speed or direction of each motor independently.
- **OLED Display (SSD1306):** The 128×64-pixel monochrome OLED display provides a visually expressive output panel for the receiver unit. Custom bitmap images representing different face expressions — happy for forward, thoughtful for still, surprised for backward, and directional for left/right — are stored in the Arduino sketch's program memory (PROGMEM) and rendered on the display corresponding to the active gesture command.
- **LM2596 Buck Converter:** The 9V battery provides the raw power input, which is stepped down to an appropriate regulated voltage by the LM2596 adjustable buck converter. This ensures stable and clean power delivery to all receiver components regardless of battery discharge state.

Table 4.2. Receiver Hardware Components and Specifications

Component	Specification	Purpose
Arduino Nano	ATmega328P, 16MHz, 32KB Flash, 2KB SRAM	Command decoding and motor control
NRF24L01 (Ultra Low Power)	2.4GHz, SPI, compact module	Wireless gesture label reception
DRV8833	Dual H-bridge, 1.5A per channel, 2.7–10.8V	N20 motor control

N20 Micro Gear Motor (×2)	3–6V DC, high gear reduction	Robot wheel actuation
OLED Display	0.96 inch, 128×64 pixels, I2C, SSD1306	Visual gesture feedback
LM2596 Buck Converter	Input 9V, Output adjustable 3.3–5V	Power regulation
9V Battery	Alkaline or Li-ion equivalent	Main power source

4.3 STANDARD COMPLIANCE

The design and implementation of this project have been carried out with reference to relevant technical standards and best practices in embedded systems, wireless communication, and machine learning deployment.

1. IEEE 802.15.4 / ISM Band Compliance: The NRF24L01 modules operate in the 2.4GHz ISM (Industrial, Scientific, and Medical) band, which is globally license-free for low-power devices. The transmission power and modulation scheme of the NRF24L01 comply with regulations governing ISM band usage in most jurisdictions.
2. I2C Communication Standard: The MPU-6050 and OLED display both use the I2C protocol as defined by the NXP I2C specification. Appropriate pull-up resistors and addressing conventions have been followed in the circuit implementation.
3. SPI Communication Standard: The NRF24L01 modules on both the transmitter and receiver use the SPI protocol in Mode 0 (CPOL=0, CPHA=0) as specified in the module datasheet, consistent with the SPI standard.
4. TensorFlow Lite for Microcontrollers: The deployed model conforms to the TensorFlow Lite Flat Buffer schema and is executed using the TensorFlow Lite for Microcontrollers inference engine embedded within the Edge Impulse Arduino library. The int8 quantization applied follows the TensorFlow Lite quantization specification.
5. Arduino Library Standard: The Edge Impulse-exported library follows the

Arduino library specification format, with appropriate library. Properties metadata, header guards, and folder structure, ensuring compatibility with the Arduino IDE library management system.

6. Motor Driver Safe Operating Conditions: The DRV8833 motor driver is operated within its rated specifications — input voltage within the 2.7–10.8V range, and per-channel current below the 1.5A continuous rating — to ensure reliable and safe motor operation.

CHAPTER 5

CONCLUSION

5.1 CONCLUSION

This project successfully demonstrated the design, training, and deployment of a TinyML-based hand gesture-controlled robot. A neural network model trained on the Edge Impulse platform achieved a validation accuracy of 97.8% across five gesture classes using spectral features extracted from six-axis IMU data captured by the MPU-6050 sensor.

The trained model was quantized to int8 precision and deployed as an Arduino library onto the ESP32 microcontroller, enabling real-time on-device inference at a 200ms inference stride without any cloud connectivity. The inferred gesture label was wirelessly transmitted via NRF24L01 radio modules to a receiver unit, where an Arduino Nano drove two N20 motors through a DRV8833 motor driver to physically manoeuvre the robot in the commanded direction. An OLED display provided simultaneous visual feedback through animated face expressions corresponding to each gesture state.

The system as a whole demonstrated that meaningful machine learning intelligence can be embedded into a low-cost, battery-powered embedded device, bridging the gap between advanced gesture recognition techniques and practical hobbyist or educational robotics applications. The use of Edge Impulse significantly streamlined the model development process, from data collection and feature engineering to training and deployment, making TinyML accessible without deep expertise in machine learning frameworks.

The minor classification confusion observed between Left and Right gesture classes, while not significantly impacting real-world performance, indicates that additional data diversity or targeted data augmentation could further improve model robustness.

5.2 FUTURE SCOPE

Several directions exist for extending and improving upon this project.

- **Expanded Gesture Vocabulary:** The current system recognizes five gesture classes. Future work could expand this to include diagonal movements, speed-varying gestures (slow tilt for slow movement, sharp tilt for fast movement), or entirely new gesture categories enabling more complex robot behaviours.

- **On-sensor Processing:** Future iterations could explore offloading inference to purpose-built TinyML accelerator chips such as the Arduino Nicla Sense ME or Sony Spresense, which offer dedicated neural processing units for even more efficient inference.
- **Longer Range Communication:** Replacing the NRF24L01 with LoRa-based modules could extend the operational range to hundreds of meters, enabling outdoor applications such as agricultural robotics or search-and-rescue robots.
- **Continuous Learning:** Incorporating on-device incremental learning or periodic model updates via OTA (Over-the-Air) firmware updates would allow the system to adapt to new users with different gesture profiles without requiring full retraining.
- **Battery Life Optimization:** Implementing deep sleep modes on both the ESP32 and Arduino Nano between inference cycles could significantly extend battery life, making the system more suitable for sustained real-world deployment.
- **Improved Visual Feedback:** Replacing the monochrome OLED with a small colour TFT display would enable richer and more expressive visual feedback, potentially including directional arrows, speed indicators, and battery status overlays.
- **IMU Calibration Routines:** Adding a built-in calibration routine that accounts for sensor bias and drift would improve consistency across different users and operating conditions, further enhancing classification reliability.

REFERENCES

1. Warden, P., & Situnayake, D. (2020). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.
2. Edge Impulse Documentation. (2024). *Getting Started with Edge Impulse*. Retrieved from <https://docs.edgeimpulse.com>
3. InvenSense. (2013). *MPU-6000 and MPU-6050 Product Specification Revision 3.4*. InvenSense Inc.
4. Nordic Semiconductor. (2008). *nRF24L01+ Single Chip 2.4GHz Transceiver Product Specification v1.0*. Nordic Semiconductor.
5. Texas Instruments. (2015). *DRV8833 Dual H-Bridge Motor Driver Datasheet (Rev. C)*. Texas Instruments.
6. Espressif Systems. (2022). *ESP32 Technical Reference Manual v5.1*. Espressif Systems.
7. Atmel Corporation. (2016). *ATmega328P Datasheet*. Microchip Technology.
8. Solomon Systech. (2008). *SSD1306 OLED Driver Datasheet*. Solomon Systech Limited.
9. Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.
10. TensorFlow Team. (2023). *TensorFlow Lite for Microcontrollers Guide*. Google. Retrieved from <https://www.tensorflow.org/lite/microcontrollers>
11. Bhatt, D., et al. (2021). CNN Variants for Computer Vision: History, Architecture, Application, Challenges and Future Scope. *Electronics*, 10(20), 2470. MDPI.
12. Zhang, Y., et al. (2018). Hello Edge: Keyword Spotting on Microcontrollers. *arXiv preprint arXiv:1711.07128*.